

Prince Sultan University

College of Computer & Information Sciences

Information Systems Department

IS499 – Senior Project

D3 – Software Requirements Specification

Mirsad (مرصاد)

A Runtime Security Firewall for AI Agents Against Prompt-Injection Attacks

Team member	Student ID	Department
Mohammed Yasser Alanssari	221110935	IS (CYS)
Bader Alshuraie	221110016	SE
Bader Abdullah Abbod Alyamini	221110197	IS (CYS)
Abdulmalik bin Saedan	221110958	IS (CYS)
Abdullah Naif Abdullah Al-Orabi	222110432	SE (AI & DS)

Supervisor Dr. Omaia Al-Omari

Semester & Academic Year Senior Project I & II – Academic Year 2026–2027

Submission Date _____

Supervisor's Signature _____

Approval Date _____

Table of Contents

Table of Contents i
List of Tables iii
1 Introduction 1
1.1 Purpose 1
1.2 Scope 1
1.3 Definitions, acronyms and abbreviations 2
1.4 References 3
1.5 Overview 3
2 Overall Description 4
2.1 Product perspective 4
2.2 Product functions 4
2.3 User classes and characteristics 5
2.4 Operating environment 6
2.5 Design and implementation constraints 6
2.6 User documentation 7
2.7 Assumptions and dependencies 7
3 Specific Requirements 9
3.1 Functional and Non-Functional Requirements 9
3.2 Use Cases 12
3.3 External Interface Requirements 16
3.3.1 User interfaces 16
3.3.2 Software interfaces 16
3.3.3 Communications interfaces 18
3.4 Data Requirements 18
3.4.1 Data entities and retention 18
3.4.2 Data integrity, protection and quality 19
4 Verification 21
4.1 Verification methods 21
4.2 Verification by requirement class 21

4.3 Traceability 23
4.4 Open verification items 23
References 24

List of Tables

Table 1: Glossary of project terms 2
Table 2: User classes and actors 5
Table 3: Operating environment 6
Table 4: Functional requirements 9
Table 5: Non-functional requirements 11
Table 6: Use-case catalogue 13
Table 7: Use-case specification UC-01 – Invoke tool via gateway 14
Table 8: Use-case specification UC-05 – Approve / deny action 15
Table 9: Use-case specification UC-08 – Manage policies 15
Table 10: Use-case specification UC-14 – Verify audit integrity 15
Table 11: User interface requirements 16
Table 12: Software interface requirements 16
Table 13: Communications interface requirements 18
Table 14: Data entities and retention 18
Table 15: Data requirements 19
Table 16: Verification of each requirement class 21

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements for **Mirsad** (مرصاد) version 1.0, a runtime security gateway that protects tool-using large language model (LLM) agents against indirect prompt injection. It is deliverable D3 of the IS499 Senior Project at Prince Sultan University and follows the information items and structure of ISO/IEC/IEEE 29148:2018 for a software requirements specification [1].

The SRS is written for four groups of readers: the project team, who build and test against it; the supervisor and examiners, who assess whether the system meets its stated objectives; testers, who derive test cases from the acceptance criteria; and future maintainers or adopting organisations, who need to know what the system promises and what it assumes about its environment. The requirement baseline is agreed at milestone M1 (20 October 2026); later changes are recorded as backlog items and reflected in a new revision of this document.

1.2 Scope

Mirsad is a self-hosted service placed between an AI agent and the tools the agent calls. It intercepts every tool call and every tool result, whether the agent uses the Model Context Protocol (MCP) or OpenAI-style function calling, and decides for each call whether to allow it, deny it or hold it for a human decision. The decision combines five mechanisms:

- a declarative YAML policy with per-tool and per-parameter constraints, rate limits and global rules;
- taint tracking, which records where values came from and stops untrusted or confidential data from reaching external destinations;
- an Arabic and English prompt-injection detector that scans tool results and removes or marks suspected instructions before the model sees them;
- human approval of high-risk actions through an installable mobile web application;
- a tamper-evident audit log in which every decision can be verified and replayed.

The product also includes an administration dashboard, a simulated enterprise workspace with a demonstration agent, and a benchmark (Mirsad-Bench) for measuring the effect of the gateway.

Mirsad does not protect the LLM against direct jailbreaks typed by its own user, does not provide network- or host-level security, does not secure model weights or training data, and does not offer high availability, horizontal scaling or multi-tenant isolation. It does not process real customer data; all demonstration and benchmark data is synthetic.

The measurable success criteria agreed in the proposal are: a detector F1 score of at least 0.85 on held-out Arabic and English test sets; a reduction in attack success rate of at least 80% relative to an unprotected agent; a benign false-positive rate of at most 5%; and a median added latency of at most 200 ms per tool call.

1.3 Definitions, acronyms and abbreviations

Acronyms follow the List of Acronyms of the final project report. Those used most often here are LLM, MCP, PWA (Progressive Web App), SSE (Server-Sent Events), JWT (JSON Web Token), HMAC, VAPID, RBAC, ASR (attack success rate) and FPR (false-positive rate). Table 1 defines the project-specific terms.

Table 1: Glossary of project terms

Term	Definition
Agent	Software that uses an LLM to plan and carry out a user's request by calling tools.
Tool, tool call, tool result	An operation the agent can invoke (for example `send_email`), one invocation with its JSON arguments, and the content the tool returns.
Upstream	A tool server (MCP over stdio or HTTP, or in-process Python functions) to which Mirsad forwards allowed calls.
Indirect prompt injection	Instructions placed by a third party in content the agent reads, such as an e-mail or web page, that the agent may follow as if they came from its user [2].
Policy	The YAML document that lists, per agent, which tools are allowed, their profiles, argument constraints, rate limits and global rules.
Tool profile	The policy attributes of a tool: trust and sensitivity of its results, sink kind, risk level, approval mode and detector settings.
Trust level	Whether a tool's results are *trusted* or *untrusted* (may contain third-party content).
Sensitivity	Classification of a tool's results: *public*, *internal* or *confidential*.
Sink	Where a call can send data: *none*, *internal*, *external* or *dynamic* (resolved from destination arguments against the organisation's internal domains).
Taint tracking	Recording identifying values (addresses, URLs, IBANs, identifiers, text fragments) from tool results with their source, and checking later calls against that record.
Decision	The gateway's verdict on a call: ALLOW, DENY or REQUIRE_APPROVAL, with machine-readable reason codes and English and Arabic explanations.
Sanitisation	Removing the spans of a tool result that the detector identified as instructions.
Spotlighting	Wrapping untrusted content in data markers with a note telling the model not to follow instructions inside it [3].
Monitor mode	A policy mode in which the would-be decision is computed and audited but every call is allowed, used to trial a policy.
Approval request	A suspended call waiting for a human decision; if no decision arrives before the timeout, it expires and the call is denied.
Audit event and hash chain	A record of one gateway event; each record's HMAC covers the previous record's hash, so modification, deletion or reordering is detectable.
Replay	Re-evaluating a stored decision with its stored policy version and inputs to check that the same action results.

Term	Definition
Misuse case	A use case performed by an attacker, linked to the use case it threatens and the security use case that mitigates it.

1.4 References

The following documents are referenced by this SRS. External sources are listed in full in the References section.

- ISO/IEC/IEEE 29148:2018, requirements engineering [1].
- Model Context Protocol specification, revision 2026-07-28 [4].
- OWASP Top 10 for LLM Applications [5].
- NCA Essential Cybersecurity Controls [6] and the Saudi Personal Data Protection Law [7].
- HMAC (RFC 2104) [8], JSON Web Token (RFC 7519) [9] and scrypt (RFC 7914) [10].
- W3C Service Workers [11], Web Application Manifest [12] and WCAG 2.2 [13].
- Project documents: the SPAP; D2 Work Plan and Budget; D4 Software Design Document; D5 Test Case Specification and Test Results; D6 Software User Guide; the engineering contract `docs/dev/BUILD_SPEC.md`; and the component guides under `docs/` (policy language, taint tracking, detector, audit and approvals, API, integration, benchmark, web UI).

1.5 Overview

Section 2 describes the product in its environment: its context, main functions, users, operating environment, constraints, assumptions and dependencies. Section 3 states the specific requirements: functional and non-functional requirements with acceptance criteria, business rules, use cases, external interface requirements and data requirements. Section 4 explains how each class of requirement is verified and where the traceability matrix is kept.

2 Overall Description

2.1 Product perspective

Mirsad is a new, self-contained component. It does not replace the agent, the LLM or the tools; it is inserted at the tool-calling boundary between them. Figure 1 shows the system boundary and the external entities. The business user gives the agent a task; the agent's tool calls pass through Mirsad, which forwards only allowed calls to the enterprise tools and returns inspected, sanitised results. An attacker who can place content in e-mails, web pages or documents that the agent reads is outside the boundary and never interacts with Mirsad directly. Approvers receive approval requests on their phones, and administrators and auditors manage policies, agents and the audit log.

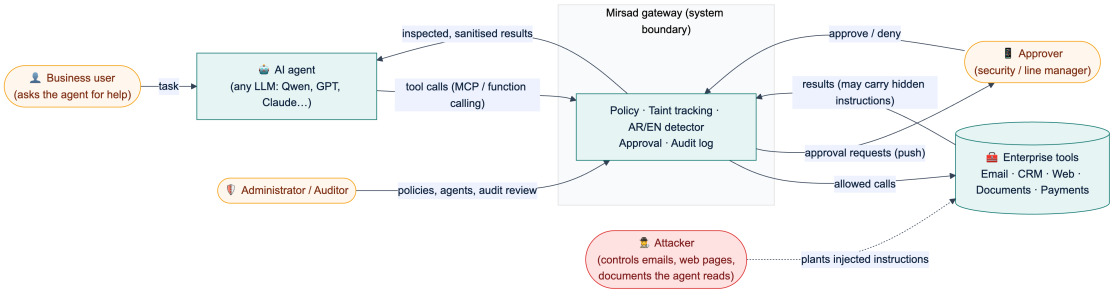


Figure 1: System context of Mirsad: the gateway between the AI agent and the enterprise tools, with the business user, approver, administrator/auditor and the attacker who plants instructions in content the agent reads

Mirsad is deployed as one service with a relational database. Agents connect to it as an MCP server, as an OpenAI-compatible chat-completions endpoint, or through a Guard API and Python SDK. Mirsad in turn connects to upstream tool servers as an MCP client. Humans use a web dashboard and an installable approver app, both served by the same service. Because the integration points are standard protocols, an organisation can place Mirsad in front of an existing agent by changing a URL or a configuration entry rather than rewriting the agent.

2.2 Product functions

The main functions, detailed as FR1–FR8 in Section 3, are:

1. **Interception (FR1).** Proxy MCP and OpenAI-style tool calls and results, and expose only the tools each agent's policy allows.
2. **Policy enforcement (FR2).** Evaluate per-tool permissions, per-parameter constraints, rate limits and global rules; combine reasons with deny-overrides; allow policies to be validated, activated and rolled back at run time.

3. **Taint tracking (FR3).** Label results by trust and sensitivity, record identifying values with their source, and escalate or block flows of untrusted or confidential data to external sinks.
4. **Injection detection (FR4).** Normalise text, scan untrusted results, and the results of tools marked for scanning such as CRM records, for instructions in English, Modern Standard Arabic, Gulf dialect, Arabizi and mixed text, and sanitise, block or flag them; spotlight untrusted content.
5. **Human approval (FR5).** Suspend high-risk calls, notify approvers in real time with a plain-language bilingual summary, and record their decisions; expire undecided requests as denials.
6. **Audit (FR6).** Append every event to an HMAC hash chain with secrets and identifiers masked; search, export, verify and replay.
7. **Administration (FR7).** Manage agents and keys, users and roles, policies and tools; show live statistics and a detector playground.
8. **Demonstration and evaluation (FR8).** Provide a simulated enterprise workspace, a demonstration agent and the Mirsad-Bench benchmark.

2.3 User classes and characteristics

Table 2 describes the classes of users and external actors. The three human roles are enforced by role-based access control (RBAC); agents authenticate with their own API keys. The personas in Section 1.5.1 of the final report (a security analyst, an AI platform engineer and a compliance officer) illustrate the three main human classes.

Table 2: User classes and actors

User class	Role	Characteristics	Main interface
Approver	`approver`	Security analyst or line manager; moderate technical knowledge; often away from a desk; reads Arabic, English or both; must decide within the approval timeout.	Approver PWA on a phone
Administrator	`admin`	Security or platform engineer; writes YAML policies, registers agents, manages users; high technical knowledge; uses the system daily during set-up and occasionally afterwards.	Dashboard, CLI
Auditor	`auditor`	Compliance or internal-audit officer; searches, verifies, replays and exports the audit log; may read policies but cannot change them or decide approvals.	Dashboard, CLI
Agent developer	agent key	Developer who connects an agent through MCP, the OpenAI-compatible endpoint or the SDK, and runs the benchmark; high technical knowledge.	Agent interfaces, SDK, documentation
AI agent	agent key	Non-human system actor that issues tool calls; treated as untrusted once it has read untrusted	MCP, `/v1` endpoints

User class	Role	Characteristics	Main interface
		content.	
Business user	none	Employee who asks the agent for help; never uses Mirsad directly but sees the explanation when an action is blocked.	Through the agent
Attacker	none	Misuse actor who controls content the agent reads; has no access to the gateway, its configuration or its secrets.	None

2.4 Operating environment

Table 3: Operating environment

Component	Environment
Gateway server	Python 3.11 or later (developed and tested on 3.12) on Linux or macOS; one Uvicorn process; CPU only, no GPU required. Default address 127.0.0.1:8700.
Database	SQLite file in the data directory (default) or PostgreSQL (the Compose file uses PostgreSQL 16).
Container deployment (optional)	Docker image and Docker Compose stack with the gateway and PostgreSQL, plus an optional profile with Ollama for the demonstration agent.
Web clients	Current desktop and mobile browsers with service workers, the Fetch streams API and notifications; Android browsers; iOS and iPadOS 16.4 or later, where push notifications require the PWA to be added to the Home Screen [14].
Web build tools	Node.js 20.19 or later to build the dashboard and approver app.
Agents	Any MCP client (stdio or streamable HTTP) or any client of the OpenAI chat-completions API with tool calling; Python programs through the SDK.
Demonstration LLM	Ollama serving Qwen2.5-7B-Instruct on the same machine or network [15], [16].

2.5 Design and implementation constraints

- **Standard protocols only.** The gateway must work with any LLM vendor (NFR2); it uses MCP and OpenAI-style JSON and no vendor-specific SDK in the decision path. The official MCP Python SDK version 2 is used, and all protocol code is kept in the adapter package so that protocol changes stay local.
- **CPU-only detection.** The detector is a character and word n-gram linear model combined with weighted rules, so that it runs within the latency budget on hardware without a GPU. Model weights are stored as NumPy arrays and JSON metadata and loaded without pickle.
- **Fail-closed behaviour.** If a decision layer, the detector or the audit log fails, the call is denied or the result blocked; a call that cannot be audited is not executed.
- **Portable persistence.** Only column types supported by both SQLite and PostgreSQL are used. Rate-limit counters are kept in memory per process, so running several gateway replicas is out of scope.

- **Bounded user input.** Regular expressions in policies are checked to limit denial-of-service risk, and request bodies are limited to 1 MB.
- **Bilingual output.** Every message that can reach an approver or administrator exists in English and Arabic, and the user interfaces support right-to-left layout.
- **OpenAI-compatible endpoint is non-streaming.** Requests with `stream=true` are rejected.
- **Regulation and ethics.** Personal identifiers are masked in logs in line with the PDPL [7]; controls are aligned with the NCA Essential Cybersecurity Controls [6]; demonstration and benchmark data is synthetic and uses reserved `.example` domains.
- **Data authorship.** Attack-labelled detector data and benchmark attack files are written and reviewed by the team, not by the AI assistant used during development, so these inputs are not produced automatically.
- **Project limits.** Two academic semesters, five students and the revised budget of 1,185 SAR set out in D2. The software is released under the Apache-2.0 licence.

2.6 User documentation

The release includes the D6 Software User Guide (installation, administration and the approver workflow), component guides under `docs/` for developers and integrators, an OpenAPI description of the REST API served at `/openapi.json`, and `--help` text for every command of the `mirsad` CLI.

2.7 Assumptions and dependencies

Assumptions. If any of these does not hold, some requirements cannot be met.

- AS-1: Agents can reach the protected tools only through Mirsad. The deploying organisation enforces this through network and credential configuration; a direct path to a tool bypasses every control.
- AS-2: The policy correctly describes each tool's trust, sensitivity, sink and risk, and lists all of the organisation's internal domains.
- AS-3: The server secret (used to sign JWTs and to derive the audit HMAC key) is kept from attackers and from insiders who can write to the database.
- AS-4: Approvers are reachable within the approval timeout (120 seconds by default); otherwise requests expire and the calls are denied.
- AS-5: Any deployment reachable from other machines terminates TLS in front of the gateway.
- AS-6: Attackers can influence only content that the agent reads; they cannot change the agent, its system prompt, the gateway or its configuration (the threat model of the final report, Section 3.2).

Dependencies.

- DP-1: The MCP specification [4] and the official MCP Python SDK version 2.

- DP-2: Ollama and the Qwen2.5-7B model for the demonstration agent and LLM benchmark runs.
- DP-3: Browser push services, including Apple's for iOS, for Web Push delivery.
- DP-4: Team-authored labelled data for detector training and for benchmark attack episodes.
- DP-5: Third-party open-source libraries, monitored by dependency vulnerability scans.

3 Specific Requirements

This section states the requirements that the delivered system must satisfy. Identifiers are used throughout the project documents: FRx.y for functional requirements, NFRn for non-functional requirements, BRn for business rules, UC-nn and MC-nn for use cases and misuse cases, IF-UI-nn, IF-SW-nn and IF-CO-nn for external interface requirements, and DR-nn for data requirements. Interface and data requirements refine the functional and non-functional requirements named in their last column, so they are traced through them.

3.1 Functional and Non-Functional Requirements

Functional requirements

Functional requirements are grouped by the eight capabilities in the proposal (FR1–FR8) and refined into testable sub-requirements. Priorities use the MoSCoW scale (M = must, S = should, C = could). Each requirement has an acceptance criterion; the test cases that verify it are listed in the requirements traceability matrix (Appendix B of the final report; deliverable D5, Section B.2).

Table 4: Functional requirements

ID	Requirement	Pri.	Acceptance criterion
FR1.1	The gateway shall act as an MCP server towards agents and an MCP client towards upstream MCP servers over stdio and streamable HTTP, forwarding `tools/list` and `tools/call`.	M	An MCP client connected through Mirsad lists and calls upstream tools; results are identical to a direct connection when the policy allows.
FR1.2	The gateway shall expose an OpenAI-compatible `/v1/chat/completions` endpoint that inspects `tool` messages before forwarding and checks returned `tool_calls`.	M	A denied tool call is removed from the response and replaced by an explanation; tool messages are sanitised.
FR1.3	The gateway shall expose a Guard API (`/v1/guard/tool-call`, `/v1/guard/tool-result`, `/v1/tools/{tool}/invoke`) and a Python SDK for frameworks that execute tools themselves.	M	SDK `guard` decorator blocks a denied call by raising `MirsadBlocked`.
FR1.4	Each agent shall only discover the tools its policy allows (least-privilege discovery).	S	`tools/list` and `GET /v1/tools` omit tools with `allow: false`.
FR2.1	The system shall load policies written in a declarative YAML language with per-agent tool entries, glob patterns, defaults, global rules and a monitor mode.	M	All shipped policies validate; invalid documents are rejected with line-numbered messages.
FR2.2	The policy engine shall evaluate per-parameter constraints (`max`, `min`, `max_length`, `enum`, `not_in`, `pattern`, `not_pattern`, `domain_in`, `domain_not_in`, `max_items`, `type`, `required`), each with a configurable violation action (deny or require approval).	M	Each operator has passing and violating test cases.

ID	Requirement	Pri.	Acceptance criterion
FR2.3	The policy engine shall enforce sliding-window rate limits per agent and tool.	M	The (n+1)-th call inside the window is denied with code `RATE_LIMIT`.
FR2.4	Reasons from all layers shall be combined with a deny-overrides algorithm, and every decision shall carry machine-readable reason codes and bilingual explanations.	M	A call with one deny reason is denied regardless of other reasons.
FR2.5	Administrators shall be able to validate, activate and roll back policy versions at run time without restarting the gateway.	S	Activation hot-reloads the policy and creates a `policy.activated` audit event.
FR3.1	Every tool result shall be labelled with a trust level (trusted/untrusted) and a sensitivity level (public/internal/confidential) from its tool profile.	M	The audit event for each inspection records both labels.
FR3.2	The taint tracker shall record identifying values (e-mail addresses, URLs, domains, IBANs, phone numbers, identifiers and text n-grams) from results, with their source tool, call and field.	M	Values embedded in JSON and in free text are both extracted.
FR3.3	The gateway shall escalate or block calls to external sinks after untrusted content was ingested, calls that carry confidential values to external sinks, and calls whose destination value originated in untrusted content.	M	The three taint rules fire on attack fixtures and do not fire when replying to a sender or e-mailing a CRM contact.
FR3.4	After an injection has been detected in a session, high-risk tools shall require approval.	S	`TAINT_AFTER_INJECTION` fires for high/critical tools in that session only.
FR4.1	Every untrusted tool result, and every result of a tool marked `scan` (for example CRM records), shall be scanned for embedded instructions in English, Modern Standard Arabic, Gulf dialect, Arabizi and mixed text, producing a score, label, language and highlighted spans.	M	Detector F1 ≥ 0.85 on the held-out Arabic and English test sets.
FR4.2	Text shall be normalised before detection (Unicode NFKC, invisible and bidirectional control characters, Arabic diacritics and letter variants, homoglyphs), and obfuscation shall itself be reported as a signal.	M	Zero-width, homoglyph and diacritic-obfuscated payloads are detected.
FR4.3	Depending on policy, detected content shall be sanitised (offending spans removed), blocked (content withheld) or flagged; untrusted content shall be spotlighted (data-marked) before reaching the model.	M	Sanitised output no longer contains the payload; spotlighting wrapper present.
FR5.1	High-risk actions (payments, deletions, bulk exports, outbound messages under taint) shall be suspended and an approval request with full context created.	M	The call blocks until a decision or timeout.
FR5.2	Approvers shall receive a real-time notification and a plain-language Arabic/English summary that explains what the agent wants to do, why it was paused and where the data came from.	M	Summary mentions the tool, destination and source of tainted data.
FR5.3	Approvers shall approve or deny from a mobile application with an optional note; requests not decided before the timeout shall expire and be denied (fail-	M	Expired requests resolve to DENY; double decisions are rejected.

ID	Requirement	Pri.	Acceptance criterion
	safe).		
FR6.1	Every intercepted event shall be appended to an HMAC-SHA-256 hash-chained audit log with secrets and personal identifiers masked.	M	Verification detects modification, deletion and reordering of events.
FR6.2	Users shall search, filter, view and export (CSV/JSON) audit events in the dashboard.	M	Filters by agent, tool, action, type, session, time and text return correct pages.
FR6.3	Any stored decision shall be replayable: the system re-evaluates it with the stored policy version and inputs and reports whether the outcome matches.	M	100% of benchmark decisions replay to the same action.
FR7.1	Administrators shall manage agents and their API keys (create, rotate, disable), users and roles (admin, approver, auditor), policies and tool registrations.	M	API keys are shown only once and stored hashed.
FR7.2	The dashboard shall show live statistics (calls, blocked, approvals, detections, latency) and a detector playground.	S	Overview reflects new events without reload (SSE).
FR8.1	The system shall ship a simulated enterprise workspace (e-mail, CRM, web, documents, payments) exposed as an MCP server and a demonstration LLM agent.	M	Demo scenarios run in protected and unprotected modes.
FR8.2	The system shall ship Mirsad-Bench, a reproducible benchmark harness with English and Arabic scenarios, ablation configurations and metric reports.	M	One command (`scripts/run_bench.sh`) reproduces the benchmark result tables of the evaluation (final report, Chapter 4; deliverable D5, Section B.5).

Non-functional requirements

Table 5: Non-functional requirements

ID	Quality attribute	Requirement	Measure / verification
NFR1	Performance	Median added latency ≤ 200 ms per tool call, excluding time waiting for a human.	Latency benchmark over 1,000 HTTP calls (p50, p95, p99).
NFR2	Interoperability / portability	Model-agnostic: no dependency on a specific LLM provider; standard protocols only (MCP, OpenAI-style JSON).	Same gateway used by an MCP client and by an OpenAI-compatible agent backed by a local model.
NFR3	Security	Agents authenticate with API keys (stored as SHA-256 hashes) and humans with JWTs and role-based access; secrets are never logged; audit integrity is verifiable; security headers, body-size limits and login throttling are applied.	Security test suite (Section 4.2.6 of the final report), redaction tests, SAST and dependency scans.
NFR4	Usability	An approver can understand and decide on a request in under 30 seconds on a phone; UI	Keystroke-Level Model estimate and task

ID	Quality attribute	Requirement	Measure / verification
		available in Arabic (RTL) and English.	walkthrough.
NFR5	Deployability	Deployable with a single `docker compose up` (or one script without Docker).	Deployment guide (Appendix E); local run script verified.
NFR6	Reliability / fail-safe	If the approver does not respond, the call is denied; internal errors in a tool never crash the gateway and are reported to the agent safely.	Expiry and executor-failure tests.
NFR7	Accountability	Every allowed, denied or approved action is attributable to an agent, session, policy version and (for approvals) a named human.	Audit event schema; replay test.
NFR8	Privacy	Personal identifiers (national ID, IBAN, card numbers) are masked in logs; demo data is synthetic.	Redaction unit tests.
NFR9	Maintainability	Modular layers with typed interfaces, ≥ 80% statement coverage of the core package, linted code, documented APIs.	Coverage report, Ruff, OpenAPI docs.
NFR10	Accessibility	Dashboard and approval app follow WCAG 2.2 AA basics: keyboard navigation, labels, contrast ≥ 4.5:1, no colour-only signals, reduced motion.	UI review checklist.
NFR11	Scalability (bounded)	Single node handles ≥ 50 concurrent agent sessions; state is in a relational database (SQLite in development, PostgreSQL in deployment).	Concurrency test in the latency benchmark.

Business rules and constraints

- BR1: Unknown agents and tools not mentioned in the policy are denied (default-deny).
- BR2: Payments and record deletions always require human approval, whatever the risk score.
- BR3: No more than 50 customer records may be exported or e-mailed in one call.
- BR4: E-mail to domains outside the organisation requires approval; data originating from untrusted content may never choose its own destination.
- BR5: An approval request expires after 120 seconds (configurable) and expiry means denial.
- BR6: Audit events are append-only; no user role can edit or delete them through the application.

3.2 Use Cases

Figure 2 shows the actors and use cases. Besides the four human roles (approver, administrator, auditor and agent developer) the AI agent is modelled as a system actor, and the attacker is modelled as a *misuse actor* following the misuse-case technique: each

misuse case is linked to the use case it threatens and to the security use case that mitigates it.

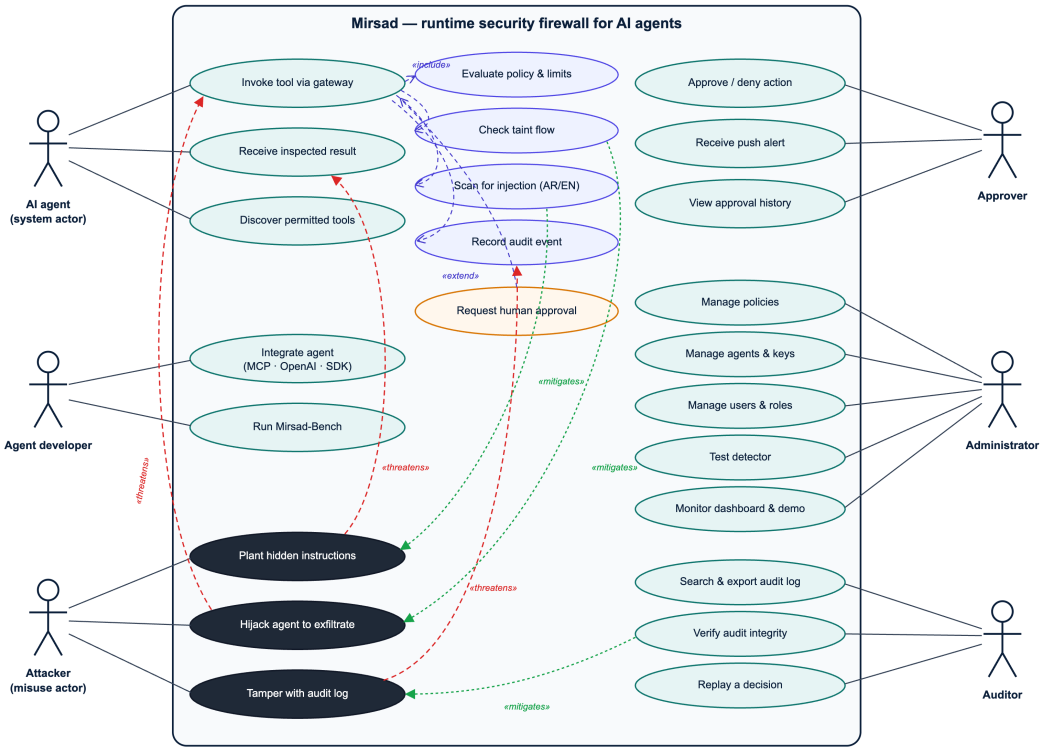


Figure 2: Use-case diagram with misuse cases (black) and the security use cases that mitigate them

Table 6: Use-case catalogue

ID	Use case	Primary actor	Summary
UC-01	Invoke tool via gateway	AI agent	The agent calls a tool; Mirsad authenticates the agent, evaluates policy, rate limits and taint, possibly asks for approval, executes the call and returns the inspected result.
UC-02	Receive inspected result	AI agent	Mirsad scans the tool result, sanitises or blocks injected instructions, labels trust and sensitivity, and returns spotlighted content.
UC-03	Discover permitted tools	AI agent	The agent lists tools and sees only those allowed for it.
UC-04	Integrate agent	Agent developer	Connect an agent through the MCP proxy, the OpenAI-compatible endpoint or the SDK.
UC-05	Approve / deny action	Approver	Review a pending high-risk action on the phone and decide.
UC-06	Receive push alert	Approver	Get a real-time notification when an action is paused.
UC-07	View approval history	Approver	Review past decisions and notes.
UC-08	Manage policies	Administrator	Edit, validate, activate or roll back the YAML

ID	Use case	Primary actor	Summary
			policy.
UC-09	Manage agents and keys	Administrator	Register agents, issue, rotate and revoke API keys.
UC-10	Manage users and roles	Administrator	Create approvers, auditors and administrators.
UC-11	Test detector	Administrator	Paste Arabic or English text and see the detector's score and highlighted spans.
UC-12	Monitor dashboard and demo	Administrator	Watch live statistics; run the demonstration agent in protected and unprotected modes.
UC-13	Search and export audit log	Auditor	Filter events and export evidence.
UC-14	Verify audit integrity	Auditor	Recompute the HMAC hash chain and locate any tampering.
UC-15	Replay a decision	Auditor	Re-evaluate a stored decision with its stored policy version and inputs.
UC-16	Run Mirsad-Bench	Agent developer	Reproduce the evaluation with scripted or LLM agents and ablations.
MC-01	Plant hidden instructions	Attacker	Place instructions in an e-mail, web page, document or CRM note the agent will read (mitigated by UC-02).
MC-02	Hijack agent to exfiltrate	Attacker	Make the agent send confidential data or money to the attacker (mitigated by UC-01 policy, taint and approval).
MC-03	Tamper with audit log	Attacker / insider	Modify or delete audit records to hide an incident (mitigated by UC-14).

Table 7, Table 8, Table 9 and Table 10 specify the four core use cases.

Table 7: Use-case specification UC-01 – Invoke tool via gateway

Field	Description
Actors	AI agent (primary); Approver (secondary, only when escalated)
Preconditions	The agent is registered and holds a valid API key; an active policy exists; the upstream tool server is reachable.
Trigger	The agent's model emits a tool call (MCP `tools/call`, an OpenAI `tool_calls` entry, or a Guard API request).
Main flow	1. Adapter authenticates the agent key and builds a `ToolCall`. 2. Policy engine resolves the tool profile, checks that the tool is allowed, evaluates parameter constraints, rate limits and global rules, and resolves the sink. 3. Taint tracker evaluates the call against the session's provenance state. 4. Gateway combines reasons (deny-overrides) and computes the risk score. 5. The decision is appended to the audit log. 6. The call is forwarded to the upstream tool. 7. UC-02 inspects the result. 8. The inspected result is returned to the agent.
Alternative flows	A1 (deny): at step 4 the action is DENY; the agent receives a bilingual explanation and the call never reaches the tool. A2 (approval): at step 4 approval is required; UC-05 is triggered and the call waits; approved → continue at 6, denied or expired → A1. A3 (monitor mode): the would-be decision is logged but the call is always forwarded.

Field	Description
Exceptions	E1 invalid key → HTTP 401, audited as authentication failure; E2 upstream error → error result returned to the agent and audited; E3 request larger than 1 MB → HTTP 413.
Postconditions	Exactly one `tool_call.decision` event (and, if executed, one `tool_result.inspection` event) exists in the hash chain for the call.
Requirements	FR1.1–FR1.3, FR2.1–FR2.4, FR3.3, FR5.1, FR6.1, NFR1, NFR3

Table 8: Use-case specification UC-05 – Approve / deny action

Field	Description
Actors	Approver (primary)
Preconditions	The approver is logged in to the approval app (role approver or admin); a request is pending.
Trigger	The gateway escalates a call (REQUIRE_APPROVAL) and publishes `approval.requested`.
Main flow	1. The app shows a notification and adds the request to the pending list with a risk badge and a countdown. 2. The approver opens it and reads the plain-language summary ("what", "why paused", "where the data came from"). 3. The approver optionally expands the raw arguments. 4. The approver taps Approve or Deny, optionally adds a note, and confirms. 5. The server records the decision, resumes the waiting call and audits `approval.decided`.
Alternative flows	A1: the request expires before step 4 → status EXPIRED, the call is denied. A2: another approver decides first → the app refreshes and shows the outcome.
Postconditions	The approval record stores the decision, the approver's username, the time and the note.
Requirements	FR5.1–FR5.3, NFR4, NFR6, NFR7

Table 9: Use-case specification UC-08 – Manage policies

Field	Description
Actors	Administrator
Preconditions	Logged in with role admin.
Main flow	1. Open the Policies page; the active YAML and version history are shown. 2. Edit the YAML. 3. Press Validate; errors are listed with line numbers. 4. Review the diff against the active version. 5. Press Activate and confirm. 6. The new version is stored, the gateway hot-swaps its policy and `policy.activated` is audited.
Alternative flows	A1: validation fails → activation disabled. A2: roll back by activating an earlier version. A3: switch to `mode: monitor` to trial a policy without enforcement.
Requirements	FR2.1, FR2.5, FR7.1

Table 10: Use-case specification UC-14 – Verify audit integrity

Field	Description
Actors	Auditor or administrator
Main flow	1. Press "Verify chain" (or run `mirsad audit verify`). 2. The server recomputes the

Field	Description
	HMAC of every event in sequence order and compares it with the stored hash and the next event's `prev_hash`. 3. The result shows the number of events checked, the head hash, and either "intact" or the first sequence number where the chain breaks.
Postconditions	An `audit.verified` event records the check.
Requirements	FR6.1, FR6.3, NFR3, NFR7

3.3 External Interface Requirements

3.3.1 User interfaces

Table 11: User interface requirements

ID	Requirement	Refines
IF-UI-01	The administration dashboard shall be served at `/` as a single-page application with the pages Login, Overview, Activity (audit log), Approvals, Policies, Agents, Users, Tools, Detector Playground, Demo Console and Settings.	FR6.2, FR7.1, FR7.2
IF-UI-02	The approver application shall be served at `/approve` as an installable PWA with a web app manifest and a service worker [12], [11]. It shall list pending requests with a risk colour and a countdown, and its detail view shall show the plain-language summary, why the call was paused, where the data came from, the raw arguments (collapsed by default) and Approve and Deny buttons that ask for confirmation.	FR5.2, FR5.3, NFR4
IF-UI-03	Both interfaces shall offer English and Arabic through a language toggle, with a right-to-left layout for Arabic.	NFR4
IF-UI-04	Both interfaces shall be operable by keyboard, label every control, meet WCAG 2.2 AA contrast, avoid colour-only signals and honour reduced-motion and light or dark preferences [13].	NFR10
IF-UI-05	The dashboard and the approver app shall update from the server event stream without a page reload.	FR5.2, FR7.2
IF-UI-06	The Policies page shall validate YAML with line-numbered errors, show the difference from the active version and require confirmation before activation.	FR2.1, FR2.5
IF-UI-07	An agent API key shall be displayed only once, when it is created or rotated.	FR7.1, NFR3
IF-UI-08	The `mirsad` CLI shall provide the commands `serve`, `init`, `policy validate`, `policy activate`, `audit verify`, `audit export`, `audit replay`, `scan`, `agent create`, `user create`, `mcp-proxy` and `detector train`.	FR2.5, FR6.1, FR6.3, FR7.1

3.3.2 Software interfaces

Table 12: Software interface requirements

ID	Interface	Requirement	Refines
IF-SW-01	MCP server for agents	Mirsad shall act as an MCP server [4] over stdio	FR1.1, FR1.4,

ID	Interface	Requirement	Refines
		(`mirsad mcp-proxy`) and streamable HTTP at `/mcp`. `tools/list` returns only the tools the agent's policy allows; every `tools/call` passes through the gateway; a denied call returns an MCP tool error whose text explains the block in English and Arabic. Agents authenticate with `Authorization: Bearer mk_...` or `X-Mirsad-Key`.	FR2.4
IF-SW-02	MCP client to upstream tools	Mirsad shall connect to upstream tool servers over stdio, streamable HTTP or in-process, as configured in `configs/upstreams.yaml`, and shall also accept plain Python functions registered as tools.	FR1.1, FR8.1
IF-SW-03	OpenAI-compatible API	`POST /v1/chat/completions` shall accept the chat-completions format, inspect every `tool` message before forwarding it, check every returned `tool_calls` entry, remove denied calls and append an explanation. The session is taken from the `X-Mirsad-Session` header or derived from the first user message. Requests are forwarded to the configured LLM base URL, by default a local Ollama server [17].	FR1.2, NFR2
IF-SW-04	Guard API	`POST /v1/guard/tool-call`, `POST /v1/guard/tool-result`, `POST /v1/tools/{tool}/invoke` and `GET /v1/tools` shall return a decision, an inspection outcome, a combined response and the visible tool list as JSON.	FR1.3, FR1.4
IF-SW-05	Python SDK	`MirsadClient` and `AsyncMirsadClient` shall provide `check_call`, `inspect_result`, `invoke`, `tools` and a `guard` decorator that raises `MirsadBlocked` when a call is denied.	FR1.3
IF-SW-06	Administration REST API	Endpoints under `/api/` shall exchange JSON, return errors as `{"detail": ...}` without echoing submitted values, and be described by OpenAPI at `/openapi.json`. Humans authenticate with HS256 JSON Web Tokens [9]; access is limited by the roles admin, approver and auditor. Agent keys are rejected on `/api/` and user tokens on `/v1/`.	FR5.3, FR6.2, FR7.1, NFR3
IF-SW-07	Database	Persistence shall go through SQLAlchemy 2.0 to SQLite or PostgreSQL, selected by the database URL, with the same code and schema on both.	NFR5, NFR11
IF-SW-08	LLM backend	The demonstration agent and the OpenAI-compatible proxy shall reach the model only through an OpenAI-compatible API whose base URL, model name and key are settings.	FR8.1, NFR2

3.3.3 Communications interfaces

Table 13: Communications interface requirements

ID	Requirement	Refines
IF-CO-01	The gateway shall serve HTTP. Deployments reachable from other machines shall be served over HTTPS through a TLS-terminating proxy or tunnel, and the gateway adds HSTS when the request arrived over HTTPS. The approver PWA requires a secure context, because service workers run only over HTTPS, with `localhost` treated as secure for development [18].	NFR3, FR5.3
IF-CO-02	Every response shall carry a Content Security Policy restricted to the server's own origin (the interactive API documentation pages have their own, wider policy), `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY` and `Referrer-Policy: no-referrer`; API responses shall carry `Cache-Control: no-store`. Request bodies over 1 MB, declared or streamed, shall be rejected with HTTP 413. Cross-origin requests shall be accepted only from configured origins.	NFR3
IF-CO-03	`GET /api/stream` shall deliver Server-Sent Events named `approval.requested`, `approval.decided` and `audit.event` with JSON data. Clients read the stream with `fetch`, so the token is sent in the `Authorization` header and never in a URL, and they reconnect with back-off. Hosting paths without SSE support, such as Cloudflare Quick Tunnels [19], are not supported for the approval channel.	FR5.2, FR7.2, NFR3
IF-CO-04	When a VAPID key pair is configured, the server shall send approval notifications through the browsers' push services. Browsers register at `POST /api/push/subscribe` using the public key from `GET /api/push/vapid` and leave at `POST /api/push/unsubscribe`; subscriptions that the push service reports as gone (HTTP 404 or 410) are deleted. Without VAPID keys Web Push is disabled and the event stream still delivers requests.	FR5.2, NFR4
IF-CO-05	Agent traffic shall use MCP over stdio or streamable HTTP, or JSON over HTTP for the OpenAI-compatible and Guard endpoints. Agent credentials shall travel only in request headers.	FR1.1–FR1.3, NFR3

3.4 Data Requirements

3.4.1 Data entities and retention

Table 14 lists the data that Mirsad stores or holds and how long it is kept in version 1.0. The relational tables are designed in detail in D4.

Table 14: Data entities and retention

Entity	Main attributes	Created by	Retention
User (`users`)	Username, salted scrypt password hash [10], role, display name, language, disabled flag	Administrator (API or CLI)	Until an administrator deletes it. Audit events and approval records keep the username as text, so deleting a user does not remove attribution.
Agent (`agents`)	Identifier, name, description, policy agent identifier, key prefix, SHA-256	Administrator	Until deleted. The full key is shown once and never

Entity	Main attributes	Created by	Retention
	hash of the key, created and last-seen times, disabled flag		stored.
Policy version (`policies`)	SHA-256 version identifier, YAML text, created at and by, active flag, comment	Administrator, or the first start-up	Kept indefinitely, because replaying a stored decision needs its exact policy version.
Audit event (`audit_events`)	Sequence number, timestamp, event type, session, agent, tool, call identifier, action, risk, policy version, redacted payload of at most 20,000 characters, previous hash, hash	Gateway and API	Append-only; no role can edit or delete events through the application (BR6). Version 1.0 has no purge function; the deploying organisation sets the retention period and archives through JSON or CSV export.
Approval (`approvals`)	Identifier, created and expiry times, status (pending, approved, denied, expired, cancelled), agent, session, call, tool, redacted arguments, decision, English and Arabic summaries, risk, decided by and at, note	Gateway	Kept with the audit log; each request and decision is also an audit event.
Push subscription (`push_subscriptions`)	User, endpoint, `p256dh` and `auth` keys, created time	Approver's browser	Until unsubscribed or reported gone by the push service.
Taint session state (memory only)	Per session: identifying values with source tool, call and field; whether untrusted content was read and whether an injection was detected	Gateway	Not persisted. At most 2,000 values per session; a session idle for 6 hours is dropped; all state is lost on restart. A minimal snapshot is stored in each decision's audit payload for replay.
Detector dataset and model (files)	JSONL records with text, label, language, variety, carrier, attack type, difficulty and source; model weights and metadata	Team; training script	Versioned in the repository; provenance documented in a datasheet [20].
Evidence reports (files under `reports/`)	Test, coverage, scan, latency and benchmark results; `results_summary.json`	Evidence scripts	Regenerated for each deliverable and kept with the release.
Secrets	Server secret (signs JWTs; the audit HMAC key is derived from it), VAPID key pair, database password	Operator	Environment variables, or a generated file in the data directory readable only by its owner; never logged or exported.

3.4.2 Data integrity, protection and quality

Table 15: Data requirements

ID	Requirement	Refines

ID	Requirement	Refines
DR-01	Each audit event shall store `hash = HMAC-SHA-256(key, prev_hash + "|" + canonical JSON of the event)`, where canonical JSON has sorted keys and no whitespace and the first event uses a previous hash of 64 zeros [8]. Verification shall report the first sequence number at which the chain breaks.	FR6.1, NFR7
DR-02	Before an event is hashed and stored, secrets (API keys, bearer tokens, JWTs, private keys and fields named like passwords, secrets or tokens) shall be removed, and payment-card numbers, Saudi national identity numbers and IBANs shall be masked. Approval arguments shall be redacted in the same way.	FR6.1, NFR8
DR-03	Passwords shall be stored only as salted scrypt hashes and agent keys only as SHA-256 hashes; no API shall return either.	NFR3
DR-04	Timestamps shall be stored as UNIX epoch seconds.	NFR7
DR-05	Demonstration and benchmark data shall be synthetic: a fictional company, reserved `.example` domains and invented phone numbers, IBANs and identifiers.	NFR8
DR-06	Detector records shall follow the schema in `docs/detector.md`. Training, validation and test splits shall be separated by authoring batch so that no test text appears in training, and the dataset build shall report a leakage check.	FR4.1
DR-07	Stored policy versions shall be immutable; activating an earlier version shall re-use its stored text and identifier.	FR2.5, FR6.3

4 Verification

4.1 Verification methods

Each requirement is verified by one or more of the four methods of ISO/IEC/IEEE 29148 [1]: **test** (an automated or scripted check with an expected result), **demonstration** (operating the system and observing the behaviour), **analysis** (measurement or calculation from data) and **inspection** (review of code, configuration or documents against a checklist). Automated tests are preferred, because they are repeated on every change. All measured values below are read from `reports/results_summary.json`, which `scripts/run_evidence.sh` regenerates from the test runs, scans and benchmarks.

4.2 Verification by requirement class

Table 16: Verification of each requirement class

Requirement class	Method	How it is verified	Current evidence
FR1 Interception and adapters	Test, demonstration	Adapter tests with real MCP clients over stdio and HTTP; OpenAI-compatible proxy and SDK tests; demonstration agent running through the gateway	78 adapter tests and 8 integration tests pass
FR2 Policy, BR1–BR5	Test	Passing and violating cases for each constraint operator; rate-limit windows; deny-overrides; scenario tests for the business rules; hot reload	265 policy tests pass
FR3 Taint tracking	Test	Rules fire on fixtures that model exfiltration flows and stay silent on benign counterparts such as replying to the sender	93 taint tests pass
FR4 Detector	Test, analysis	Unit tests of normalisation, rules, sanitising and spotlighting; precision, recall and F1 on held-out English and Arabic test sets	227 detector tests pass; F1 English [pending], Arabic [pending] (target 0.85)
FR5 Approval	Test, demonstration	Approval-service tests (expiry, double decision), API tests, web tests, approval round trip on a phone	Approval tests within the 240 audit-area tests; 194 web tests (38 Vitest suites in 10 test files) pass
FR6 Audit, BR6, DR-01, DR-02	Test	Tamper tests (modify, delete, reorder); query and export tests; redaction tests; replay of every decision after each benchmark run	272 of 272 decisions replayed to the same action in one scripted run and 272 of 272 in the other
FR7 Administration, IF-SW-06	Test	API tests including role checks for each endpoint; UI tests	185 API tests pass
FR8 Demonstration and benchmark	Test, demonstration	Demonstration scenarios in protected and unprotected modes; benchmark runs for every defence configuration	72 demo and 68 benchmark tests pass; 48 benign tasks per scripted run

Requirement class	Method	How it is verified	Current evidence
NFR1 Performance	Analysis	Latency benchmark: 1,000 calls, sequential and with concurrent sessions, server in a separate process	Median added latency 2.7 ms sequential, 166 ms with 50 concurrent sessions (target 200 ms)
NFR2 Interoperability	Test, demonstration	Adapter tests with MCP clients and with the OpenAI-compatible proxy; an LLM agent that reaches a local model through the OpenAI-compatible API and calls tools through the gateway	Adapter tests (see FR1); LLM smoke run with 4 tasks, audit chain intact
NFR3 Security, IF-CO-01–05	Test, analysis	Security test suite; static analysis; dependency scans; penetration test (T6.4)	45 security tests pass; 0 Bandit issues; 0 vulnerable Python packages of 124; 0 npm findings
NFR4 Usability, IF-UI-02, IF-UI-03	Analysis, inspection	Keystroke-Level Model estimate of the approval task [21], a task walkthrough and a user study	Partially verified: KLM estimate completed (final report, Section 4.2.8: 21.9 s and 28.8 s reading the summary, 44.1 s reading every reason of a multi-reason request); user study pending
NFR5 Deployability	Demonstration	Native installation and start-up; `docker compose up` on a machine with Docker	Partially verified: native installation used for all measurements; Docker deployment not yet executed
NFR6 Reliability	Test	Expiry leads to denial; tool failures are reported safely; failures of the audit log or detector fail closed	Within the API and audit-area tests
NFR7 Accountability, DR-04	Inspection, test	Audit event schema review; replay	See FR6
NFR8 Privacy, DR-02, DR-05	Test, inspection	Redaction unit tests; review of demonstration data	Within the audit-area tests
NFR9 Maintainability	Analysis	Statement coverage of the core package; linting	94.4% coverage (target 80%)
NFR10 Accessibility, IF-UI-04	Inspection	WCAG 2.2 checklist review of both interfaces [13]	Partially verified: contrast and accessible names tested automatically; checklist review pending (T6.1)
NFR11 Scalability	Analysis	Concurrent sessions in the latency benchmark	50 sessions with 0 errors
Objective 3 benchmark targets	Analysis	Mirsad-Bench with scripted and LLM agents against an unprotected baseline	Benign false-positive rate 0.0% (target at most 5%); ASR reduction [pending] (target at least 80%)
DR-03, DR-06, DR-07	Test	Authentication tests; dataset build leakage check; policy-store tests	Within the audit, detector and API tests

Detector F1 and attack success rate are **not measured yet**. Both need attack-labelled data written by the team (Section 2.5). Once the team adds the data as described in `docs/detector.md` and `bench/scenarios/attacks/README.md`, `make detector` produces the F1 values, and the benchmark, which already executes attack episodes when attack files exist, produces the attack success rate. Until then these values are shown as pending.

The benign measurements are precise about what they cover. In the scripted benchmark with full protection, 0.0% of benign calls were denied automatically and 17.6% were escalated to a human. With an approver who accepts legitimate requests, utility was 100%; if every escalation were denied, it would fall to 75%. The LLM run covers only 4 tasks and is a smoke test, not a utility estimate.

4.3 Traceability

The requirements traceability matrix (RTM) links every functional and non-functional requirement to the test cases that verify it and to their results. It is maintained with the test specification in D5 and reproduced in Appendix B of the final report. The acceptance criteria in the requirement tables of Section 3.1 define what each test must show; external interface and data requirements are traced through the FR and NFR identifiers in their "Refines" column. A change to a requirement after the M1 baseline is recorded as a backlog item and updated in this SRS, in the RTM and in the affected tests in the same sprint.

4.4 Open verification items

1. Train and evaluate the detector on the team's labelled data (`make detector`), then fill the FR4 row.
2. Add the team's attack files and measure the attack success rate against the unprotected baseline.
3. Run the Docker Compose deployment on a machine with Docker (NFR5).
4. Run the approval-time user study (NFR4; the Keystroke-Level Model estimate is reported in Section 4.2.8 of the final report) and the WCAG checklist review (NFR10).
5. Repeat the latency benchmark with the trained detector model, and run the LLM agent on the full task set rather than the smoke test.

References

[1] Systems and Software Engineering — Life Cycle Processes — Requirements Engineering, ISO/IEC/IEEE 29148:2018, Nov. 2018.

[2] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in Proc. 16th ACM Workshop Artif. Intell. Secur. (AISec '23), Copenhagen, Denmark, 2023, pp. 79-90, doi: 10.1145/3605764.3623985.

[3] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kıcıman, "Defending against indirect prompt injection attacks with spotlighting," in Proc. Conf. Applied Machine Learning in Information Security (CAMLIS 2024), CEUR Workshop Proceedings, vol. 3920, Arlington, VA, USA, Oct. 2024, pp. 48-62. Accessed: Oct. 11, 2026. [Online]. Available: https://ceur-ws.org/Vol-3920/paper03.pdf

[4] Model Context Protocol, "Specification," protocol revision 2026-07-28. Accessed: Oct. 11, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28.

[5] OWASP GenAI Security Project, "OWASP Top 10 for LLM Applications 2025," OWASP Foundation, Nov. 2024. Accessed: Oct. 11, 2026. [Online]. Available: https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/

[6] National Cybersecurity Authority, Essential Cybersecurity Controls (ECC-2:2024), Riyadh, Saudi Arabia, 2024. Accessed: Oct. 11, 2026. [Online]. Available: https://cdn.nca.gov.sa/api/files/public/upload/86e09090-44e4-481f-bc28-355673607654_ECC--2024-EN.pdf

[7] Kingdom of Saudi Arabia, Personal Data Protection Law, Royal Decree No. M/19 of 9/2/1443H (16 Sep. 2021), Council of Ministers Resolution No. 98 of 7/2/1443H, amended by Royal Decree No. M/148 of 5/9/1444H (Mar. 2023). Accessed: Oct. 11, 2026. [Online]. Available: https://laws.boe.gov.sa/BoeLaws/Laws/LawDetails/b7cfae89-828e-4994-b167-adaa00e37188/1

[8] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," RFC 2104, Feb. 1997, doi: 10.17487/RFC2104.

[9] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, May 2015, doi: 10.17487/RFC7519.

[10] C. Percival and S. Josefsson, "The scrypt password-based key derivation function," RFC 7914, Aug. 2016, doi: 10.17487/RFC7914.

[11] M. Chintala and Y. Yanagisawa, Eds., "Service Workers Nightly," W3C Candidate Recommendation Draft, Sep. 17, 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://www.w3.org/TR/service-workers/

[12] M. Cáceres, D. Murphy, and C. Liebel, Eds., "Web Application Manifest," W3C Working Draft, Oct. 9, 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://www.w3.org/TR/appmanifest/

[13] A. Campbell, C. Adams, R. B. Montgomery, M. Cooper, and A. Kirkpatrick, Eds., "Web Content Accessibility Guidelines (WCAG) 2.2," W3C Recommendation, Oct. 5, 2023

(updated Dec. 12, 2024). Accessed: Oct. 11, 2026. [Online]. Available: https://www.w3.org/TR/WCAG22/

[14] B. Eidson and J. Simmons, "Web Push for Web Apps on iOS and iPadOS," WebKit Blog, Feb. 16, 2023. [Online]. Available: https://webkit.org/blog/13878/web-push-for-web-apps-on-ios-and-ipados/. Accessed: Oct. 11, 2026.

[15] Ollama, "ollama/ollama," GitHub repository, MIT License. [Online]. Available: https://github.com/ollama/ollama. Accessed: Oct. 11, 2026.

[16] Qwen Team, "Qwen2.5-7B-Instruct," model card, Hugging Face. Accessed: Oct. 11, 2026. [Online]. Available: https://huggingface.co/Qwen/Qwen2.5-7B-Instruct.

[17] Ollama, "OpenAI compatibility," Ollama Docs. [Online]. Available: https://docs.ollama.com/api/openai-compatibility. Accessed: Oct. 11, 2026.

[18] MDN contributors, "Service Worker API," MDN Web Docs, Mozilla. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API. Accessed: Oct. 11, 2026.

[19] Cloudflare, "Quick Tunnels," Cloudflare Docs. [Online]. Available: https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/do-more-with-tunnels/trycloudflare/. Accessed: Oct. 11, 2026.

[20] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III, and K. Crawford, "Datasheets for datasets," Commun. ACM, vol. 64, no. 12, pp. 86–92, Dec. 2021, doi: 10.1145/3458723.

[21] S. K. Card, T. P. Moran, and A. Newell, "The keystroke-level model for user performance time with interactive systems," Commun. ACM, vol. 23, no. 7, pp. 396–410, Jul. 1980, doi: 10.1145/358886.358895.